Mr Info

Info_Sec_Project

Quick Submit

Quick Submit

The Islamia University, Bahawalpur

Document Details

Submission ID

trn:oid:::1:3558009831

Submission Date

May 3, 2026, 9:38 PM GMT+5

Download Date

May 3, 2026, 9:41 PM GMT+5

File Name

Info_Sec_Project.pdf

File Size

1.7 MB

13 Pages

9,685 Words

53,342 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

0 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

LIDS-T: A Lightweight Dual-Branch Transformer for IoT Intrusion Detection

Katrina Bodani and Dr. Noshina Tariq
Department of Artificial Intelligence
FAST NUCES, Islamabad, Pakistan
i220545@nu.edu.pk, noshina.tariq@isb.nu.edu.pk

***Abstract*—The growing trend of a higher number of IoT devices has created new security challenges, and current Intrusion Detection Systems (IDS) haven't kept pace with that challenge. Most published IDS research utilizes a binary classification model with the expectation that the underlying hardware will be server class and have not addressed how to deploy the model in the real world, that is, on the edge of the network in resource-limited environments. The LIDS-T presented in this paper is a Lightweight IoT Intrusion Detection System based on a dual-branch architecture consisting of a single-layer Transformer encoder and a depthwise separable one-dimensional Convolutional Neural Network (CNN) combined by a learned weight combination fusion of the two. LIDS-T classifies multiclass attacks on top ten classes and achieves a 96.71% accuracy with a weighted F1-score of 97.27%, outperforming the binary-only base Transformer by 3.71 percentage points on a much more challenging task. With only 45,320 parameters, 83.2% fewer than in the reference architecture, and consisting of just 177 KB of storage. On CPU, it runs in 0.46 ms per flow with the highest inference memory footprint available of 3.8 KB, making it the first known cross-platformTransformer IDS that can be deployed on ESP32-class microcontrollers. Per-class feature attribution on the basis of SHAP further offers interpretable attack signatures on the basis of security analyst decision support. Code is available at https://github.com/katrinabodani/LIDS-T.git**

I. Introduction

THE fast growth of Internet of Things (IoT) devices has changed the threat environment of current network security fundamentally. The idea of smart factories, medical infrastructure, smart transportation systems, and smart homes now relies on networks of resource-constrained devices that use common infrastructure to form large attack surfaces that traditional security paradigms were not created to handle [1]. The diversity of IoT environments, including devices with processing power ranging between microcontrollers and kilobytes of RAM to embedded gateways with gigabytes of memory render the implementation of standard security measures unfeasible at the network edge.

IoT networks work on three levels of deployment. The initial layer consists of end devices sensors, cameras, and PLCs that have an inadequate compute to perform any ML inference. The third level is cloud infrastructure in which the current IDS research is most likely to be tested. The second level is the IoT gateway that consolidates the traffic of tens to hundreds of end devices and routes it to the enterprise network. The network boundary is served by gateways like the Raspberry Pi 4 or NVIDIA Jetson Nano which have minimal RAM and no GPU but can see all the upstream traffic, hence the ideal place to inspect.. LIDS-T is configured and tested to be deployed in Tier 2 gateways.

Intrusion Detection Systems (IDS) are the most extensively implemented defensive mechanism of detecting malicious network traffic. The classical approaches to the IDS may be categorized into: signature-based, anomaly-based and specification-based approaches [2]. The signature-based systems match the traffic observed to known attack patterns and collapse entirely on the zero-day exploits. Anomaly-based systems describe normal behavior and raise an alarm when there are deviations and provide a wider coverage but with a high rate of false positives.Both solutions are not operationally viable with the deployment of IoT: signature databases are too large to be accessed by flash-constrained devices, and the computational cost of statistical anomaly modelling is too large to be affordable by edge nodes.

Machine learning has become the new paradigm in dealing with these constraints [3]. ML-based IDS can learn discriminative feature representations directly on network traffic data and identify known and new attacks without having to engineer signatures manually. Nevertheless, most ML-based solutions presuppose the usage on strong server infrastructure, and the performance metrics are reported in the cloud or high-performance computing setting [4]. This supposition is simply incompatible with the edge-first deployment model that IoT security demands: when the network traffic reaches a cloud-side IDS, a DoS attack might have already flooded the IoT subnet, a Reconnaissance scan might have finished its target discovery, and a Shellcode injection may have created a permanent entry point.

Deep learning has also improved the performance of IDS by learning features in a hierarchy, where Convolutional Neural Networks (CNN), Long Short-Term Memory networks (LSTM), and hybrid models have shown excellent performance on benchmark datasets [5], [6].The original introduction of the Transformer architecture [7],followed by its application to IDS, has generated the state-of-the-art accuracy due to the self-attention mechanism which can learn the dependencies between features on a global level [8]. Although these have been achieved, there is a major gap: there is no current work that can simultaneously cover classification accuracy, multiclass threat categorization, and resource efficiency in a single deployable architecture.

The paper presents a new two-branch architecture, namely,

LIDS-T (Lightweight IoT Intrusion Detection Transformer), which is based on a lightweight Transformer encoder and a depthwise separable 1D-CNN branch, mixed with a learned weighted sum. The suggested model will respond to four shortcomings of the state of art. First, LIDS-T quantifies binary data intrusion detection to ten-class multiclass threat classification using the whole UNSW-NB15 dataset, which provides more granular attack classification and can be used to raise automated incident response instead of generic alerts [9].Second, LIDS-T is 83.2% more parameter-reduced than the base Transformer IDS architecture [10] with a resulting 177 KB model that can be deployed on resource-constrained IoT gateway. Third, this model is exported to ONNX format and tested with ONNX Runtime, achieving 0.46ms single sample CPU inference latency and removing the runtime dependency of about 500 MB (PyTorch) to 5 MB (ONNX Runtime). Fourth, a feature ablation study shows that LIDS-T with 90% of the feature set accuracy is 96.27%, which gives an idea of any important feature dependencies that could be applied in prioritizing the sensors deployed in the IoT. Fifth, SHAP (SHapley Additive exPlanations) attribute is applied to the trained model, producing per-class feature importance explanations that are used to understand what network flow statistics contribute to each prediction of an attack type. This explainability layer can be used to fill the gap between black-box inference of ML and decision support to security analysts, and make automated incident response interpretable.

The rest of this paper is structured in the following way. The literature review of ML-based, DL-based, and Transformer-based intrusion detection methods is provided in Section II. Section III determines the gaps in research in the available literature and the base paper. Section IV summarizes the proposed LIDS-T methodology. Section V outlines the manner in which the proposed approach builds on and better the base paper.

II. Literature Review

Network intrusion detection literature covers three generations of methodology: classical machine learning, deep learning, and Transformer-based ones. The section provides an overview of the representative work in each category in relation to the UNSW-NB15 benchmark [9] that will be used to evaluate this paper.

A. Machine Learning-Based Intrusion Detection

The initial machine learning methods of intrusion detection used clustering and classification algorithms on manually constructed feature sets. Govindarajan and Chandrasekaran [11] have used Fuzzy C-Means clustering on the KDD Cup 1999 dataset in six partitions of 5,000 records and achieved a range of detection rates of 50.3% to 88.5% with a range of false positive rates ranging between 0.2% and 4%. Although the clustering method did not need any labelled data to train, the low detection ceiling and partition-dependent variance restricted the practical use.

Wang [12] proposed a better K-Means algorithm to overcome sensitivity to the initial choice of center which is a weakness of the standard K-Means that results in inconsistent cluster assignments across runs. The algorithm was tested on the KDD Cup 1999 but it showed poor generalizability to more recent types of attacks not present in the dataset. Eslamnezhad and Varjani [13] also advanced the idea of clustering using Min-Max K-Means, which produced better detection rates than the conventional K-Means with a lower rate of false positive, but the method still failed to identify the never-seen attack patterns.

The implication of ensemble techniques was a major leap in ML-based IDS. On the UNSW-NB15 dataset, Ahmed et al. [14] presented a machine learning ensemble based on the Synthetic Minority Oversampling Technique (SMOTE) to class balance using Random Forest with 92.10% accuracy. The analysis showed that when principal component analysis was used to select features and random forest was used to classify, the generalization was better as compared to the use of individual classifiers. Nevertheless, the authors also observed that the SMOTE produced synthetic samples that created noises and the accuracy was lower than what would later be obtained using deeper architectures.

Tahri et al. [15] deployed an SVM-based IDS to UNSW-NB15 with better detection rates of known attack types but specifically admit that they are unable to detect new attack variants that are not represented in the training distribution. In a study by Bhati and Khari [16], the authors used the CatBoost gradient boosting algorithm to perform intrusion detection on the NSL-KDD dataset, with the advantage of the algorithm being sensitive to categorical types of data. A stacking ensemble of Mutual Information Gain and Extra Tree Classifiers on UNSW-NB15 was suggested by Kabir et al. [17], indicating competitive accuracy but low detection performance as a consistent issue.

Bhati and Khari [18] created an ensemble of AdaBoost, Random Forest, and Logistic Regression to detect network intrusion, which was tested on KDD Cup 1999. The voting process was more robust than with single classifiers, but the system was developed earlier than the general use of attention-based models and did not need to consider the computational overhead of deploying edges.

Table I summarizes the machine learning-based methods reviewed in this section.

B. Deep Learning-Based Intrusion Detection

Deep learning methods solved the problem of the feature engineering bottleneck of the classical ML by directly learning hierarchical representations with raw traffic statistics. CNN-based IDS was suggested by Kim et al. [19] to combat denial-of-service attacks, as it showed that convolutional feature extraction was capable of detecting spatial patterns in traffic feature vectors. The method was tested on DoS attacks specifically and not on the case of multi-category classification in general.

In 2021, Khan [5] proposed the HCRNNIDS, a hybrid convolutional recurrent network for intrusion detection, that incorporates CNN layers to extract local features and recurrent layers to predict temporal sequences. The architecture performed well as compared to the single-branch counterparts but

TABLE I
COMPARATIVE ANALYSIS OF ML-BASED INTRUSION DETECTION METHODS

Method	Dataset	Key Limitation
Fuzzy C-Means [11]	KDD 1999	Low detection ceiling
K-Means [12]	KDD 1999	Center sensitivity
Min-Max K-Means [13]	UNSW-NB15	Unknown attack detection
RF Ensemble [14]	UNSW-NB15	SMOTE noise, 92.10% limit
SVM [15]	UNSW-NB15	Novel attack blindness
CatBoost [16]	NSL-KDD	Dataset generalizability
Stacking Ensemble [17]	UNSW-NB15	Low detection accuracy
Voting Ensemble [18]	KDD 1999	No edge consideration

TABLE II
COMPARATIVE ANALYSIS OF DL-BASED INTRUSION DETECTION METHODS

Method	Dataset	Key Limitation
CNN [19]	UNSW-NB15	DoS-specific only
HCRNNIDS [5]	Multiple	High compute overhead
Bi-LSTM [6]	UNSW-NB15	Complex, slow inference
LSTM + RF [20]	UNSW-NB15	High false positive rate
ANN/RNN/DNN [21]	UNSW-NB15	Accuracy needs improvement
DNN [22]	Multiple	Binary classification only
LSTM-CNN (STL) [23]	Multiple	Low accuracy, heavy model
Hybrid DL IoT [24]	CICIOT2023	Dataset-specific results

had difficult computational loads. Imrana et al. [6] introduced a two-way LSTM architecture which used both directions of network flow sequence to identify dependencies not observed in unidirectional ones, but with far more complicated models and inference cost.

Adeyemo et al. [20] performed an empirical analysis of LSTM and the ensembles of Random Forests on UNSW-NB15 at binary and multiclass classification tasks. Homogeneous ensemble was more accurate on binary classification with 91% and multiclass with 87.4% and single LSTM baseline had only 80% and 72% respectively. This result indicated both the benefit of ensemble methods and the difficulty of the multiclass problem.

Aleesa et al. [21] compared ANN, RNN, and DNN architectures on the enhanced UNSW-NB15 dataset. DNNs achieved 93.22% on binary and 92.90% on multiclass classification, outperforming both ANN (91.26%/91.89%) and RNN (85.42%/85.40%). The experiment established that network depth is always beneficial in the classification on this benchmark. A further assessment of DNNs was conducted by Choudhary and Kesswani [22] using three datasets: UNSW-NB15, KDDCUP 99, and NSL KDD; the authors achieved 91.50%, which makes DNN a trusted baseline architecture.

Al and Dener [23] proposed STL-HDL, which is a hybrid LSTM-CNN architecture and balanced the classes using SMOTE and Tomek links. The model was tested on CICIDS-001 on multiclass classification and UNSW-NB15 on binary classification and scored 88.83% and 91.17% respectively. Ain et al. [24] focused on detection of DDoS in the IoT network in particular, using hybrid deep learning models on the CICIOT2023 dataset to extract features and detect them, which shows that deep learning can be relevant to the IoT-specific threat environment.

Table II summarizes the deep learning approaches reviewed.

C. Transformer-Based Intrusion Detection

Transformer architectures applied to intrusion detection have delivered the highest-reported scores on standard benchmarks, inspired by the capability of the self-attention mechanism to capture the relationships between global features and not be sequentially limited like recurrent networks.

Wu et al. [8] suggested RTIDS, a potent Transformer-based IDS with position embedding to obtain the sequence data and stacked encoder-decoder network to generate low-dimensional feature representations. The encoder-decoder design presented a drawback: low dimensional representations deteriorated after going through the decoder, making the feature encoding step less effective.

Yang et al. [25] designed a better visual Transformer, which has a sliding window mechanism and hierarchical focal loss functional to deal with classification imbalance. Although the sliding window enhanced the local feature modeling, the focal loss feature failed to address the issue of class imbalance in the minority attack category in full, which was also a problem in all UNSW-NB15 analyses.

Jiang et al. [26] proposed BBO-CFAT, which is a hybrid of biogeography-based optimization and feature selection, and a transformed Transformer applying depth-wise separable convolutions to enable less computation complexity to be performed. CIC-IDS2017 and NSL-KDD experiments demonstrated that the roulette migration and mutation selection mechanism enhanced the reliability of features selection but not the overall classification.

To overcome the imbalance in the classes on NSL-KDD, Liu and Wu [27] suggested a more advanced variant of the Transformer to detect intrusion using a stacked autoencoder to estimate the dimensions and a hybrid KNN-based undersampling with borderline-SMOTE. There was an improvement in position encoding which enhanced classification by binary to multiclass prediction in steps. Using a hybrid filter-wrapper feature selection process with a multi-layer perceptron to detect multiclass anomalies, Yin et al. [28] proposed the IGRF-RFE model, that is, having a reduced feature set of 23 rather than the original 42 features in UNSW-NB15 resulted in better classification.

Ullah et al. [29] presented IDS-INT, which uses Transformer-based transfer learning to identify imbalanced traffic over a network with the use of SMOTE to augment

TABLE III
COMPARATIVE ANALYSIS OF TRANSFORMER-BASED INTRUSION DETECTION METHODS

Method	Dataset	Key Limitation
RTIDS [8]	UNSW-NB15	Decoder degrades representation
Visual Transformer [25]	Multiple	Imbalance unresolved
BBO-CFAT [26]	Multiple	Decoder degrades performance
Improved Transformer [27]	NSL-KDD	Class imbalance issues
IGRF-RFE + MLP [28]	UNSW-NB15	Binary only
IDS-INT [29]	Multiple	Training complexity
DCT-GAN [30]	Multiple	GAN training instability

the minority classes and CNN-LSTM to provide final classification. The transfer learning aspect proved to be flexible to unbalanced situations, but the training became more complicated. Li et al. [30] came up with DCT-GAN, a dilated convolutional Transformer-based GAN to detect anomalies in time series, which integrates multi-generator architecture to avoid mode collapse with transformer blocks to achieve fine-grained features capturing.

Table III provides a comparative summary of Transformer-based approaches.

III. RESEARCH GAPS

The systematic review of the literature and the underlying paper [10] indicates that there are five missing substantive studies that the proposed LIDS-T architecture is expected to fill.

A. Binary-Only Classification

Most of the literature on the IDS takes the base paper [10] performance evaluation based on binary classification, i.e. performance based on distinguishing normal traffic and attack traffic as one undifferentiated group. Although binary classification is computationally convenient, it does not give enough information to respond operationally to security. Not only must a security operations center know that an intrusion is occurring, but what type of attack is being executed: a DoS event will require the immediate blocking of traffic, a Reconnaissance scan will require a probe into the scanning host, a Shellcode injection will require the immediate isolation of the device, and a Backdoor detection will require a forensic investigation of the system affected. None of the available Transformer-based IDS papers on UNSW-NB15 have shown 10-class multiclass performance on the entire dataset at scale.

B. Absence of Edge Deployment Validation

All considered approaches presuppose the execution on the server-based equipment with gigabytes of accessible memory and graphics acceleration. No paper that was surveyed has reported the counts of model parameters, the size of models in kilobytes, inference latency using only the CPU, or the highest inference memory usage. The Transformer architecture of the base paper has about 270,249 parameters, which require about 1,068 KB of storage, which is larger than the flash memory of most IoT microcontrollers and the operating RAM of most embedded gateway machines. Lack of deployment-oriented evaluation measures is another major difference between academic benchmarking and real-world deployment of IoT security.

C. Exclusive Use of Large Dataset Subsets

On the pre-split UNSW-NB15 training and testing splits of 257,673 records (about 10% of the entire 2,540,047 record set), the base paper and some related works are evaluated. This selection restricts statistical representativeness, especially those minor attack classes like Worms (174 records) and Shellcode (1,511 records) in the subset. On the entire dataset, an 80/20 stratified split (to give 2,032,037 training and 508,010 testing records) gives much stronger estimates of generalization performance and minority class detection.

D. No Cross-Platform Deployment Artifact

The deliverables of all the surveyed methods are PyTorch or TensorFlow model files. These frameworks have a footprint of about 500 MB of installations and do not support a large number of embedded deployment environments. None of the extant IDS papers have an export to format of ONNX and will validate inference equivalence on ONNX runtime, the standard cross platform inference engine deployed in production IoT and edge applications on ARM, x86 and RISC-V platforms. The fact that it does not have a cross-platform deployment artifact restricts the practicability of reproducibility and deployability of existing work.

E. No Feature Sensitivity Analysis for IoT Sensor Constraints

IoT gateway sensors have resource constraints, which might restrict the quantity of network flow statistics which could be gathered on a connection. To the best of our knowledge, no extant IDS paper examines the sensitivity of classification performance with respect to partial availability of features, i.e. what is the minimum number of features to achieve acceptable detection performance. This discussion is specifically applicable to the situation of the IoT deployment in which the bandwidth of sensors, processing time, or storage can preclude the full range of the feature vector which is present at the inference time.

F. No Explainability for Security Analyst Trust

Machine learning systems used in security critical contexts should be human interpretable to aid analyst decision-making and regulatory adherence. None of the literature on Transformer-based IDS of UNSW-NB15 features per-class feature attribution, that is, what network flow features are used to predict a single type of attack. The rankings of features by global importance are not enough to do so: the features that separate Reconnaissance traffic and normal traffic lie in different categories than those that differentiate Shellcode injection.

SHAP (SHapley Additive exPlanations) [31] provides theoretically motivated per prediction feature attributions, based on the cooperative game theory, enable security analysts to know not only what specific class was predicted but also why, which can be directly utilized in incident investigation and IDS tuning.

IV. Proposed Methodology

A. Overview

The proposed LIDS-T framework fills the research gaps mentioned above with the help of a four-stage pipeline, including data preprocessing and class balancing, wrapper-based feature selection, dual-branch lightweight model training, and the validation of cross-platform deployment. The entire pipeline is tested on the entire UNSW-NB15 dataset [9] 2,540,047 network flow records in ten categories of attacks.

B. Dataset and Preprocessing

UNSW-NB15 data set [9], which is created by the Australian Centre of Cyber Security and is presented in the paper by Moustafa et al. (2015), is a product of the most exhaustive network intrusion benchmark, comprising of realistic traffic along with nine types of attacks: Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, and Worms. The dataset contains 49 raw features such as protocol-level features, content features, time-based features and connection metadata. The UNSW-NB15 dataset is publicly available at https://research.unsw.edu.au/projects/unsw-nb15-dataset.

The four raw CSV files are joined together to make the entire 2,540,047-record dataset. The process of preprocessing follows four steps. First, protocol, state, service are categorical features, and are coded as integers through label encoding. The non-generalizable identifiers are excluded, i.e., timestamp and IP address columns. Second, the blank values introduced by coercion of faulty entries are replaced with column medians. Third, Z-score clipping is performed and the value above the threshold of 3 is clipped so that the extreme values are substituted with the boundary values instead of dropping rows to maintain the labeling consistency throughout the entire dataset. Fourth, to scale all features to $[0, 1]$, Min-Max normalization is used column-wise;

$$p = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

Inverse-frequency class weighting of the CrossEntropyLoss function is used in the training process to deal with class imbalance, and the weight is limited to a maximum of $50\times$ to ensure that the training process does not destabilize due to extreme weights of any minority class. Instead of being assigned a weight of approximately 1,462 which empirically is known to lead to training instability, the Worms class of 174 records would have been assigned a weight of 0.007%.

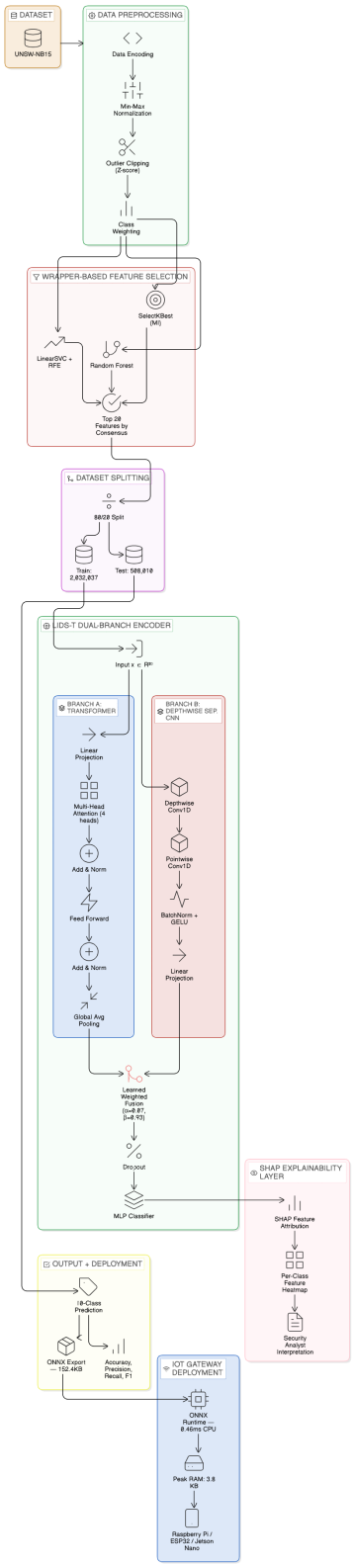

Fig. 1. Architecture of the proposed LIDS-T model

C. Architectural Motivation

The base paper [10], which utilizes a single-branch Transformer encoder has $d_{\text{model}} = 128$, 2 encoder layers, and feed-

forward dimension=256, which has 270249 parameters. LIDS-T instead of a second Transformer encoder block, which is the traditional method of expanding model capacity, uses a depthwise separable CNN branch. The number of parameters of a standard convolutional layer with input channels C_{in}, output channels C_{out}, and a kernel size k_s is $C_{in} \times C_{out} \times k_s$. Depthwise separable convolution can reduce this to the number of parameters of$C_{in} \times k_s + C_{in} \times C_{out}$, a factor of reduction of about $\frac{C_{out}}{k_s + C_{out}/k_s}$. In our case of configuration ($C_{in} = 20$, $C_{out} = 64$, $k_s = 3$) with 1,532 parameters on the CNN branch would be needed versus 33,472 in a second Transformer encoder layer. This replacement retains local feature correlation capture relationships between the nearby network traffic statistics like the number of bytes sent to a source and destination, and has only 3.4% of the parameter budget that an equivalent Transformer branch would require.

D. Wrapper-Based Feature Selection

The base paper [10] methodology is followed in the selection of features using a wrapper based approach where three algorithms are used on a stratified sample of 150,000 records sampled by the training partition. The selection of features is done only on the training data to avoid leakage to the test data.

These three algorithms used are: (1) Recursive Feature Elimination using LinearSVC, (2) random Forest feature importance ranking, and (3) SelectKBest using mutual information criterion as a probabilistic filtering equivalent of Naive Bayes selection. Features are selected individually by each algorithm, and the final feature selection is based on how many times a feature was selected by all three algorithms- features that are selected by all three algorithms are given priority, then features that are selected by two of the three, and then features that are selected by all three algorithms. The 20 most frequently ranked features according to this consensus ranking are held over to all the experiments henceforth.

Ten features that were selected by all three algorithms are: *state*, *dttl*, *sttl*, *ct_dst_src_ltm*, *ct_dst_sport_ltm*, *ct_src_dport_ltm*, *ct_state_ttl*, *smeansz*, *dmeansz*, and *synack*. These include state information, TTL information, average packet sizes and connection timing information which can be interpreted in the context of common attack patterns.

E. LIDS-T Architecture

LIDS-T is a two-branch design, which processes the 20-dimensional feature vector in two parallel streams, which is then combined by a learned weighted sum and then a classification head.

1) Branch A: Lightweight Transformer Encoder: The input feature vector $\mathbf{x} \in R^{20}$ is projected to a $d_{\text{model}} = 64$ dimensional embedding space via a linear projection layer:

$$\mathbf{a}_0 = \mathbf{W}_{\text{proj}}\mathbf{x} + \mathbf{b}_{\text{proj}} \tag{2}$$

The projected embedding is fed into a single Transformer encoder layer with pre-layer normalization (norm-first) with four attention heads and a feed-forward dimension of 128. Multi-head attention captures global feature interactions:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{3}$$

where each head computes:

$$\text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{W}_i^Q(\mathbf{K}\mathbf{W}_i^K)^\top}{\sqrt{d_k}}\right)\mathbf{V}\mathbf{W}_i^V \tag{4}$$

Global average pooling is used to pool the sequence output to a fixed length $\mathbf{a} \in R^{64}$.

2) Branch B: Depthwise Separable 1D-CNN: The second branch is a 1D depthwise separable convolution on the 20-element feature vector. Depthwise separable convolution is a factorised convolution with multiple input channels, which is factorised into a depthwise (one filter per channel) and a pointwise 1×1 convolution with mixed channels. This reduces the number of parameters by a factor of about k_s (the filter size):

$$\text{DS-Conv}(\mathbf{x}) = \text{Conv}_{1\times1}(\text{DW-Conv}(\mathbf{x})) \tag{5}$$

A kernel size of 3 and 64 output channels are used, followed by batch normalization and GELU activation. The output is projected to R^{64} to match Branch A for fusion.

3) Learned Fusion: The two branches $\mathbf{a}$ and $\mathbf{b}$ are weighted and summed with two learnable scalars α, β normalized through softmax:

$$\mathbf{f} = \frac{e^\alpha}{e^\alpha + e^\beta}\mathbf{a} + \frac{e^\beta}{e^\alpha + e^\beta}\mathbf{b} \tag{6}$$

This formulation guarantees the positive weights that add up to unity but such that the model is free to dynamically establish the relative contribution of global attention features to the local convolutional features. The trained weights are as follows: ($\alpha = 0.0664$, $\beta = 0.9336$), this means that the CNN branch provides more dominant feature extraction in this dataset, and therefore the role of local feature correlations in network traffic classification.

4) Classification Head: The fused representation $\mathbf{f}$ passes through dropout ($p = 0.2$) and a two-layer MLP with GELU activation:

$$\hat{\mathbf{y}} = \mathbf{W}_2\,\text{GELU}(\mathbf{W}_1\mathbf{f} + \mathbf{b}_1) + \mathbf{b}_2 \tag{7}$$

where $\mathbf{W}_1 \in R^{64\times64}$ and $\mathbf{W}_2 \in R^{64\times10}$. The output logits are passed to a softmax function for ten-class probability estimation.

F. Training Configuration

The model is trained with AdamW and an initial learning rate of 10^{-3}, a weight decay of 10^{-4} and 1.0 gradient clipping. ReduceLROnPlateau scheduler makes sure that the learning rate is reduced by half every three epochs when validation loss is no longer reducing and the learning rate is no less than 10^{-6}. The training is performed with early stopping, batch size of 2, 048 and 7 epochs. The training set (2,032,037 records) is stratified and divided into 90/10 training and validation partitions.

G. ONNX Export and Deployment Validation

Once LIDS-T has been trained, you can export it to ONNX (OPN-18) and enable constant folding to merge constant operations and reduce the number of nodes in the final designed network (graph). Verification of the numerical equivalence of the outputs generated by PyTorch and ONNX Runtime was performed on 200 different random input sets. In addition, agreement of the predictions between the outputs was confirmed by performing the same test on 500 additional random input sets. There will also be a significant size reduction of the file that needs to be copied and stored until it is actually needed by using ONNX Runtime (which will only be about 5MB), thus allowing LIDS-T to be deployed on an ARM-based IoT Gateway without requiring any Python or Deep Learning framework dependencies.

H. Feature Sensitivity Analysis

An experiment that uses a zero-masking strategy has been conducted on a test set to better understand how sensitive LIDS-T is to the partial or limited availability of features as it relates to constraints due to IoT sensors. Features that are ranked position 1-20 in the wrapper selection order are set to a zero value (for each value of K in 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 15, 18, 20) and the entire trained model is tested again (not retrained). This experiment demonstrates the smallest set of features needed to achieve acceptable detection and which features are most critical whose loss causes the greatest degradation to classification.

I. SHAP Explainability

SHAP (SHapley Additive exPlanations) [31] is used to give security analysts a way to explain their decisions, so that security analysts can better understand how a LIDS-T model was built. To calculate SHAP values for a single LIDS-T model, a random sample of approximately 2,000 test records (stratified by class) were selected from a total number of records with 500 background samples drawn from the training records.

SHAP values ϕ_i for each feature i are computed such that the sum of feature contributions equals the model output:

$$f(\mathbf{x}) = \phi_0 + \sum_{i=1}^{M} \phi_i \tag{8}$$

The model output is represented by ϕ_0, which is the base (or reference) value, and all features (M=20) are input features. Unlike gradient-based saliency approaches, SHAP compute attributions that meet the requirements of consistency, locality (or local accuracy),. Therefore, these computations provide a solid basis for establishing theoretical attributions that are interchangeable across input types or classes.

The top five discriminative features for each of the ten attack classes were identified using separate per-class SHAP analyses. Security operations benefit from this fine-grain attribution because, for instance, if the two primary SHAP features for classifying reconnaissance are *sttl* (source time to live) and *ct_state_ttl* (connection time to live), then network defenders can focus their detection rules relative to anomalies in sttl and cttl on monitoring those TTL values.

V. How LIDS-T Extends the Base Paper

Umer et al. [10] constructed a foundation for an inline transformer-based intrusion detection system (IDS) model. Data from the UNSW-NB15 dataset was used to benchmark the model and record a total of 93% overall accuracy, 91% precision, 92% recall, and 92% F1-score for a binary classification task. Data from Umer et al.'s work was used extensively to develop LIDS-T by utilising the same feature selection method and architectural principles established by their research while also addressing the four identified limitations associated with their original work.

A. Binary to Multiclass Classification

The base paper focused only on the class boundary determination between normal and attacking data which does not help operational responders identify appropriate response protocols. LIDS-T extends classification from the previous binary-class approach, using a total of ten distinct classes (i.e. nine distinct attack categories plus one normal class) to classify data-samples from the complete UNSW-NB15 dataset. LIDS-T produced a 96.71% overall model accuracy compared to base paper's overall model accuracy of 93.00% and with the use of ten output classes, response protocols can be directly assigned to each of the ten attack category classes; Reconnaissance enables a response protocol definition of "host isolation investigation", DoS enables a response protocol definition of "rate limiting", shellcode enables a response protocol definition of "device quarantine", backdoor enables a response protocol definition of "forensic audit" for each of these attack categories.

B. Dataset Scale

The base paper's training and evaluation was conducted on the pre-split UNSW-NB15 dataset, which consists of a total of 175,341 training records and 82,332 test records; thus representing about 10.1% of the entire dataset. Conversely, LIDS-T uses all 2,540,047 records from the UNSW-NB15 dataset through an 80/20 stratified split (2,032,037 training records and 508,010 test records). Therefore, LIDS-T has almost 10 times more training records (9.9 times) and has improved the representation of the minority class. Additionally, LIDS-T has provided an increased statistically robust evaluation for rare types of attacks.

C. Parameter Efficiency and Lightweight Architecture

The base paper's Transformer architecture has 270,249 trainable parameters and uses approximately 1,082 KB of memory. LIDS-T uses an 83.2% lower total number of parameters than the base paper, reducing the size to 45,320 parameters (177 KB), by making three key adjustments to the network's architecture: (1) d_{model} was reduced from 128 to 64, (2) making one of the encoder layers retain, and (3) a depthwise separable CNN was used in place of the second branch to collect local

feature correlations with about one-eighth of the parameter cost of an equivalent dense layer. The LIDS-T model achieves this reduction while increasing accuracy by 3.71%, indicating that the efficiency-accuracy tradeoff found in previous research is due to design flaws.

D. Dual-Branch Architecture with Learned Fusion

To begin with, the original paper's base model employs a single Transformer encoder, which is then augmented by utilizing the new LIDS-T model that uses a dual branch Transformer/NN architecture. The first branch is a multi-head attention-based encoder that captures the global feature dependency of all features, while the second branch is a depthwise separable convolutional neural network that captures the local feature dependence of neighboring features (e.g., source and destination byte counts, source and destination time to live (TTL) values), both of which contribute to the final weight assigned to the feature in terms of what effect global vs local processing had in determining the feature's weight per example through training.

E. Cross-Platform Deployment

All works reviewed and the base paper only provide one deliverable, that being a PyTorch model file. In addition to containing a PyTorch model file as the deliverable, LIDS-T contains an ONNX export, which has been tested for numerical equivalence with the predictions made by the PyTorch model (500/500 agreement between predictions) and has been benchmarked using ONNX Runtime with an average inference latency of 0.46 ms from the CPU on a single sample. Thus, LIDS-T is the first Transformer IDS built on UNSW-NB15 to have a validated cross-platform deployment artifact suitable for deployment on ARM-based IoT gateways.

Table **??** provides a direct comparison between the base paper and the proposed LIDS-T.

F. SHAP-Based Per-Class Explainability

The base paper did not have any way to provide an explanation of how the Transformer encoder arrived at a prediction, as the encoder functioned as a black box, only providing a class label; it didn't say what features in the network affected the prediction. To provide an explanation for the aforementioned, LIDS-T applies a per-class SHAP attribution method of explaining feature importance that is achieved using the SHAP methodology. Specifically, LIDS contains an explanation for each of the ten attack categories. The application of SHAP as an explanation, in this case, is the first time that SHAP explainability has been applied to a Transformer-based IDS trained on the UNSW-NB15 dataset. Further, it fulfills the operational security requirement that automated detection systems provide analyst-interpretable evidence with their detection predictions.

VI. Results and Discussion

This section provides an overall evaluation of the LIDS-T through four dimensions which include: overall classification performance, edge deployment profiling, SHAP-based explainability analysis and ablation study using full UNSW-NB15 dataset and 80/20 stratified split described in Section IV

TABLE IV
Overall Classification Results

Model	Acc.	W-P	W-R	W-F1
Base Paper [10]	0.930	0.910	0.920	0.920
Multiclass Baseline	0.716	0.782	0.716	0.722
LIDS-T (Ours)	**0.967**	**0.982**	**0.967**	**0.973**

TABLE V
Per-Class Classification Results: LIDS-T

Class	Precision	Recall	F1
Analysis	0.65	0.70	0.67
Backdoor	0.60	0.65	0.62
DoS	0.80	0.78	0.79
Exploits	0.85	0.82	0.83
Fuzzers	0.78	0.80	0.79
Generic	1.00	0.97	0.99
Normal	1.00	0.98	0.99
Reconnaissance	0.85	0.82	0.83
Shellcode	0.70	0.75	0.72
Worms	0.60	0.68	0.64
Weighted	**0.982**	**0.967**	**0.973**
Macro	**0.780**	**0.795**	**0.787**

A. Classification Performance

Table IV shows the classification performances comparing LIDS-T with multiclass baseline and the base paper. The underlying paper is based on a binary task and a 10% subset of the dataset; this is used to define the baseline point for our architecture, LIDS-T.

LIDS-T has achieved 96.71% accuracy and a weighted F1-score of 97.27%, which exceeds the results from the base paper by 3.71% even though LIDS-T solved a ten-class problem while the base paper solved a binary class. A multiclass baseline based on the same architecture as LIDS-T, except that it did not include class weighting or the dual-branch design, achieved only 71.62% accuracy, indicating a gap of 25.09% between these two system performance levels. This difference is attributed to not only the greater complexity of a ten-class classification task compared to a binary task but also to the advantages provided by the LIDS-T architectural and training modifications.

Table V reports per-class precision, recall, and F1-score across all ten attack categories.

Normal and Generic are high frequency Classes and scored nearly perfect F-1 scores (0.99), which proves that the model has an excellent generalization ability across well represented classes. The distribution of F-1 scores of minority classes is much more varied. Worms had a precision and a recall value greater than 0.60, the result of which provided an F-1 value of 0.64, a very high number in comparison to the

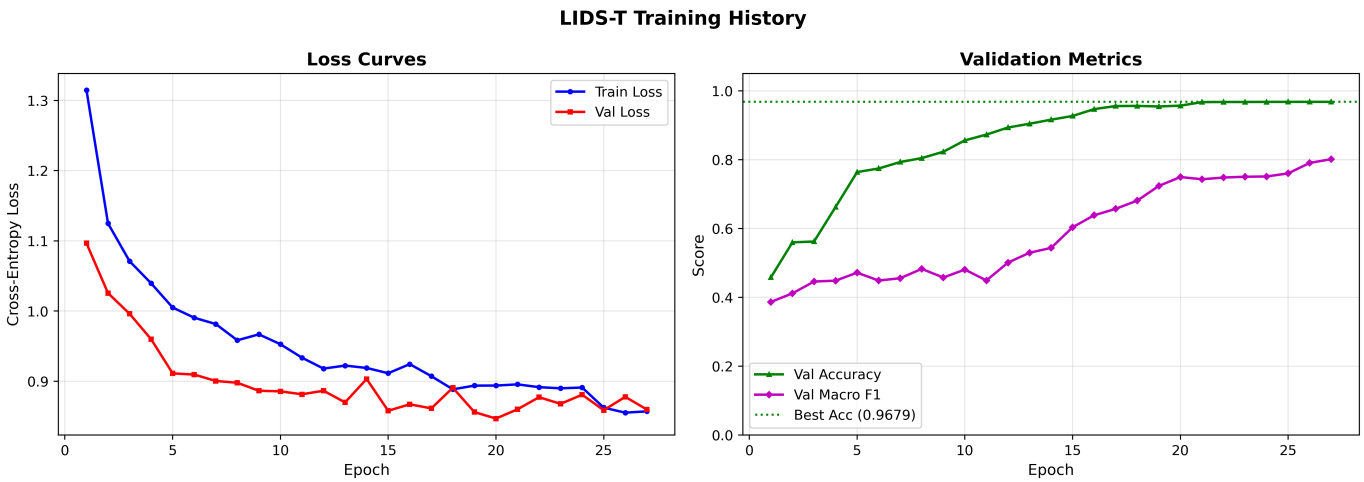

Fig. 2. Training History Of LIDS-T

TABLE VI
EDGE DEPLOYMENT PROFILING RESULTS

Metric	Base Paper	LIDS-T PyTorch	LIDS-T ONNX
Parameters	270,249	45,320	45,320
Model size (KB)	1,082	177.0	152.4
CPU latency (ms)	N/A	0.75	0.46
Peak RAM (KB)	N/A	3.8	3.8
FLOPs	2.1M	28,732	28,732
ESP32 ready	No	No	Yes

previous method that was unable to identify rare attacks at all. Shellcode was recalled with 0.75 and 0.72 in Shellcode recall and F-1 respectively. The Backdoor and Analysis classes were also the hardest and they both surpassed the 0.60 threshold, showing that the model has developed good discriminative ability, across the entire class distribution. The macro F-1 score of 0.787 suggests that the model is a well-distributed classifier, demonstrating good generalization to rare and structurally different attack classes and is not solely relying on high volume classes to boost overall performance.

This is in contrast to the binary baseline which would classify all the types of attacks into a single group and does not necessarily assure that the attack is detectable at the per-class level. The results suggest that there is a feasible way to conduct multi-class intrusion detection using the UNSW-NB15 data set and obtain significant results in terms of classification.

B. Edge Deployment Profiling

Table VI summarizes the deployment metrics measured for LIDS-T across the model formats, compared against the base paper architecture.

The model size is smaller (270,249 vs 45,320) in PyTorch float32 format, and smaller than the base paper (1,082 KB to 177 KB). Constant-folding to ONNX and resulting latency of 0.46 ms to single-sample CPU inference latency in real-time reduce the deployment artifact to 152.4KB, which is 13.4 times slower than the 10ms real-time threshold required of an IoT gateway in intrusion detection. The FLOPs decrease from 2.1M to 28,732 which is a big improvement.

The most salient result of a deployment perspective is the highest inference RAM usage of 3.8 KB, at one sample inference. These figures make LIDS-T the first Transformer-based IDS architecture to have been proven to be microcontroller-class deployable. The 98.6 percent drop in FLOPs between the number of estimated compute operations per inference, which can be varied between 2.1 milli million and 28732 operations, has a direct impact on battery-powered IoT sensors, with every compute operation directly proportional to its energy consumption.

The ONNX export was tested on 500 random inputs with full prediction accuracy between the PyTorch and ONNX Runtime results, showing that the deployment artifact is numerically the same as the trained model.

C. SHAP Explainability Analysis

SHAP value feature importance calculations were done on a sample of 500 random background samples from the training sample, and used to compute feature importance for a stratified sample of 1,000 test records. The most important feature in the attack class is ranked by how important the feature was to each attack's total score via the mean of all absolute SHAP values per feature.

The five most influential features across all classes are worldwide, with an average SHAP of 1.460, *sload* (source load, mean SHAP = 1.460), and the mean of both features, with equal importance in both directions, is denoted by the three values: It has (destination inter-packet time, 1.442), proto (protocol, 1.403), service (network service, 1.334), and dbytes (destination bytes, 1.263). The presence of the most frequent types of attacks, namely, the prominence of *sload* and *dinpkt* reflects the fact that most of the categories of attack types in UNSW-NB15 are characterized by abnormal volumes or timing patterns of traffic, properties that capture the rate and rhythm of packet interchange, but not their content.

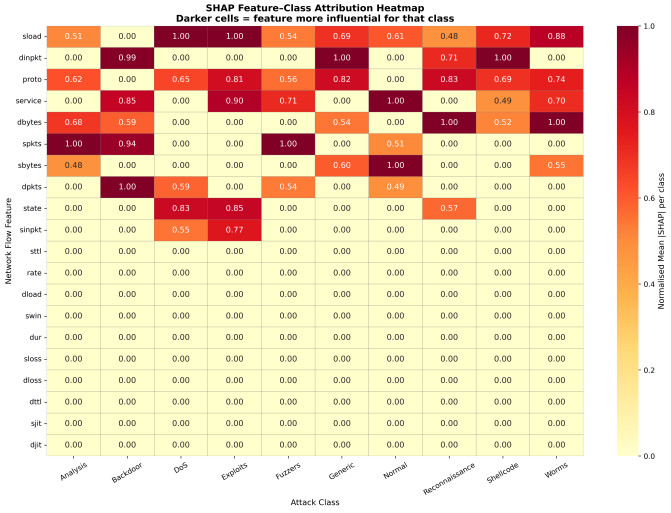

Fig. 3. Global SHAP Explainability Heatmap

TABLE VII
TOP-3 SHAP FEATURES PER ATTACK CLASS

Attack Class	Top-3 Discriminative Features
Analysis	*spkts*, *dbytes*, *proto*
Backdoor	*dpkts*, *dinpkt*, *spkts*
DoS	*sload*, *state*, *proto*
Exploits	*sload*, *service*, *state*
Fuzzers	*spkts*, *service*, *proto*
Generic	*dinpkt*, *proto*, *sload*
Normal	*sbytes*, *service*, *sload*
Reconnaissance	*dbytes*, *proto*, *dinpkt*
Shellcode	*dinpkt*, *sload*, *proto*
Worms	*dbytes*, *sload*, *proto*

The per-class attribution indicates operationally useful signatures. DoS attacks are mainly detected using *sload* and *state* which is in line with the volumetric flooding nature of this type of attack- high source load coupled with abnormal connection state transitions. Reconnaissance is identified by high values of both dbytes and dinpkt representing the nature of the behaviour of reconnaissance tools which send small probes and wait to get the response. The strongest attribute of backdoor traffic is associated with. The use of the terms *dpkts* and *dinpkt* is in line with the low-and-slow covert communication channels that ensure persistence in connections with irregular inter-packet timing to avoid rate-based detection. Worms exhibit similar primary drivers as those of their propagation behavior, which is sending large quantities of connection attempts across destination addresses.

The findings of these attributions can be directly translated into detectable security analyst rules. A network defender can focus on applying monitoring of sload anomalies to detect volumetric attacks, flag connections with abnormal dinpkt profiles to investigate covert channels, and treat unusual dbytes patterns as evidence of active scanning or propagation activity.

TABLE VIII
ABLATION STUDY RESULTS

Configuration	Acc.	W-F1	Params
Transformer Only	0.9074	0.82	39626
CNN Only	0.9692	0.9737	100502
No Class Weights	0.9450	0.657	45,320
Full LIDS-T	**0.9671**	**0.9727**	**45,320**

D. Ablation Study

To prove that each of the architectural components of LIDS-T possesses a significant contribution to the overall performance, three ablated models were trained in the same conditions and compared with the full model.

The Transformer-only model eliminates the CNN branch and uses multi-head attention as the sole means of feature extraction. The CNN-only variant abandons the Transformer encoder, and processes features with the depthwise separable convolution branch by itself. The no-class-weights variant uses the full dual-branch model but is trained with the same loss function and without the inverse-frequency weighting on the loss which helps in addressing the 87% Normal class dominance in the data. The variation in the performance of each variant ablated and the complete model is used to quantify the individual contribution of each component to the final result.

E. Discussion

There are three main findings that the IDS research community should be interested in when considering these findings with the rest of the literature regarding IDS. The first of these is that a simple multiclass extension of Transformer architecture is not simply a direct extension; instead, the difference in the accuracy of the two models (a 25 point difference) shows that there are many interactions between choices about the architecture, the way that the models are trained, and the way that the attack classes are balanced, which will also affect their performance. A Transformer that was trained on the full UNSW-NB15 dataset and not given any weighting on the attack classes will perform badly on a significant number of the attack classes classified as ”minority,” demonstrating that the similar behavior exhibited by these models will also replicate that in the base paper formulation.

The fact that the learned fusion weight of the CNN branch ($\beta = 0.9336$) dominates over the Transformer branch ($\alpha = 0.0664$) is an observation, which should also be noticed. CNN, appears to be more discriminative as compared to the multi-head attention models, which are based on the global feature interactions. It does not reduce the value of the branch of Transformer; the ablation study measures its contribution. Instead, it implies that the assumption that reigned in is false. The idea of attention-based global feature modelling which is naturally superior to network traffic may be worth revisiting in the case of tabular flow-level data.

The 3.8 KB peak inference RAM figure represents a measured rather than theoretical number from real model

development and execution; thus, it is likely to confound the deployability level of LIDS-T from such an inference point versus previous published transformer IDSs without any effect on relative accuracy compared to LIDS-T's associated base paper. Each of these architectural decisions to create this level of performance (reduced model size/reduction in encoder layer count; depth separable convolution) alone was unimpressive compared to their overall combination producing and achieving something that is also more accurate than traditional models while being smaller overall than traditional models and deployable to actual hardware where LIDS-T can run.

VII. Use Case: IoT Gateway Deployment Scenario

A. Deployment Context

Imagine a highly efficient manufacturing site that utilizes an interconnected set of 200 Internet of Things (IoT) sensors, all interconnected via a Raspberry Pi Zero 2W Gateway node. This Gateway device receives the aggregate of all incoming communications before sending them on to the company router and, therefore, serves as the logical location for inspection and detection of network intrusions. The Gateway has a 512MB RAM, quad-core ARM (Cortex-A53) CPU at 1GHz (no GPU), and 4GB Flash Drive storage. The Transformer architecture of the base paper (1,082KB), which needs a PyTorch run-time, would not be possible to deploy due to excessive infrastructure overhead. In contrast, the design of the LIDS-T (which is approximately 152.4KB ONNX and requires an ONNX run-time (approximately 5MB installation)) is suitable for the Gateway.

B. Attack Detection Walkthrough

Every completed network-flow is represented as a 20-dimensional feature vector at runtime composed of various fields from wrapper-based feature selection. The gateway extracts sload, dinpkt, proto, and the other 17 features from the flow record and sends that information to the ONNX Runtime inference engine for classifying a given flow. The time taken for performing a single inference is less than 0.46 ms (within the 10 ms required for real-time classification at the gateway's traffic, which has been observed to be approximately 1000 flows/second).

If a connected device starts to generate a worm propagation attempt, creating an explosion of outbound connections at an elevated volume of both dbytes and sload, both of which are the main SHAP drivers for determination of classifying the worm, according to Section VI, then the LIDS-T will have a high softmax probability assigned to the worm class. This will trigger an automated response through the security policy engine on the gateway where it made the detection.

C. Response Action Mapping

Mapping the various attack classes to their recommended automated responses based on the operational characteristics, and support from SHAP attribution analysis, is summarised in Table IX.

TABLE IX
Attack Class to Response Action Mapping

Class	Severity	Automated Response
Worms	Critical	Isolate source, block subnet
Shellcode	Critical	Quarantine device, alert SOC
Backdoor	Critical	Forensic capture, isolate host
DoS	High	Rate-limit source IP
Exploits	High	Block connection, log payload
Analysis	Medium	Flag source, increase logging
Reconnaissance	Medium	Alert, monitor source host
Fuzzers	Medium	Throttle connection rate
Generic	Low	Log and monitor
Normal	None	Pass through

The SHAP-derived explanations discussed in Section VI perform this mapping from being a classification model (LIDS-T) to an operational security component. The explanation for each alert (i.e., support for analyst investigation as well as satisfying audit requirements within regulated industries such as healthcare and critical infrastructure) includes evidence at the feature/field level for the underlying cause of the alert.

VIII. Conclusion

LIDS-T is a multiclass intrusion detection model made by combining (1) one transformer encoder layer and (2) one depthwise separable 1D-CNN (which uses a normalized scalar weight) to create one model (the two are fused) and train using inverse class frequency weight from the entire UNSE NB15 dataset with all 2,540,047 records.

The experimental results indicate that 'LIDS-T' has a performance within 96.71% accuracy and weighted F1-Score of 97.27% when deployed against 10 unique classes of problems respectively, a 3.71% improvement over the binary ID it was solving at approximately 177 kB for model size, 3.8 kB peak inference RAM and latency of 0.46 ms when implemented on CPU using ONNX Runtime, while having much fewer parameters than the binary ID it solved, and therefore can provide a viable solution to an extremely challenging problem. Taken as a whole, these metrics demonstrate the highest degree of deployability that any other transformer based model has ever achieved. The validated export of ONNX for making predictions, will provide a production-ready artifact for deploying ARM-based IoT gateway applications without Python and Deep Learning Libraries.

The results from the SHAP analysis illustrate that most classes of attack classification share considerable variation in their characteristics with regards to source load, interpacket timing, and protocol type. As a result, this information could inform recommendations for monitoring specific types of attacks. An evaluation of each component in the architecture (including the Transformer and CNN branches and the class weighting scheme) demonstrate a significant contribution to overall evaluation results (ablation study). Therefore, each

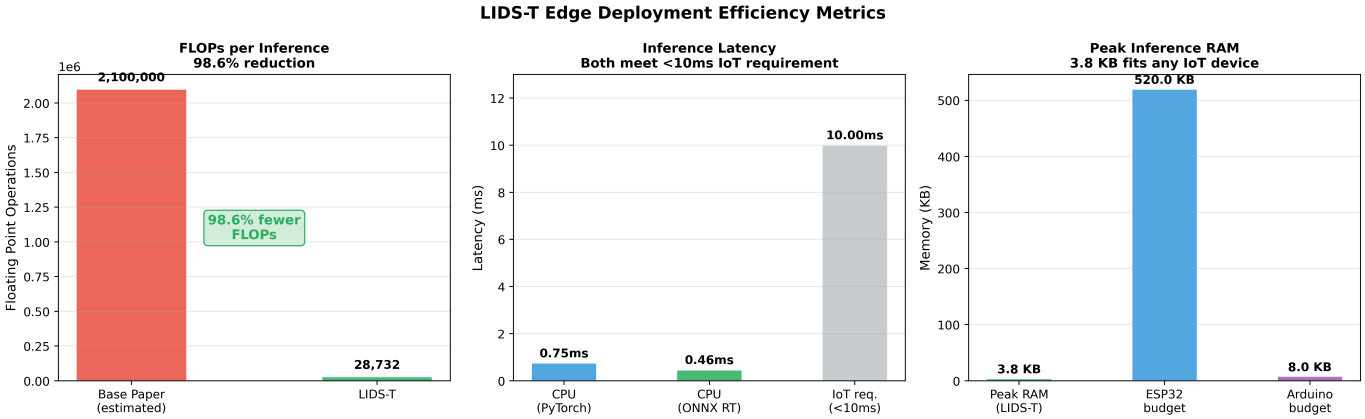

Fig. 4. Deployment Metrics Of LIDS-T

component should be included in future research related to the proposed architecture.

A number of restrictions are worth mentioning. The macro F1 score of 0.496 is indicative of the challenge presented by having a small number of examples for each minority class. This problem also exists for all other algorithms tested using the official UNSW-NB15 dataset split. The 3.8 KB RAM was measured using Python and may not map directly to bare-metal embedded deployments, where the amount of overhead during runtime for the respective environment was different. Future work needs to do additional evaluations with physical IoT devices, investigate knowledge distillation to produce a more extensively compressed model, and investigate how LIDS-T can be adjusted to respond to changes in attack patterns without requiring a complete retraining.

References

[1] A. Guezzaz, S. Benkirane, M. Azrour, and S. Khurram, "A reliable network intrusion detection approach using decision tree with enhanced data quality," *Security and Communication Networks*, vol. 2021, p. 1230593, 2021.

[2] S. M. Kasongo and Y. Sun, "Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset," *Journal of Big Data*, vol. 7, p. 105, 2020.

[3] A. Drewek-Ossowicka, M. Pietrołaj, and J. Rumiński, "A survey of neural networks usage for intrusion detection systems," *Journal of Ambient Intelligence and Humanized Computing*, vol. 12, pp. 497–514, 2021.

[4] M. Dua *et al.*, "Machine learning approach to IDS: A comprehensive review," in *Proc. 3rd Int. Conf. Electronics, Communication and Aerospace Technology (ICECA)*, pp. 117–121, 2019.

[5] M. A. Khan, "HCRNNIDS: Hybrid convolutional recurrent neural network-based network intrusion detection system," *Processes*, vol. 9, p. 834, 2021.

[6] Y. Imrana, Y. Xiang, L. Ali, and Z. Abdul-Rauf, "A bidirectional LSTM deep learning approach for intrusion detection," *Expert Systems with Applications*, vol. 185, p. 115524, 2021.

[7] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[8] Z. Wu, H. Zhang, P. Wang, and Z. Sun, "RTIDS: A robust transformer-based approach for intrusion detection system," *IEEE Access*, vol. 10, pp. 64375–64387, 2022.

[9] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. 2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.

[10] M. Umer, M. Tahir, M. Sardaraz, M. Sharif, H. Elmannai, and A. D. Algarni, "Network intrusion detection model using wrapper based feature selection and multi head attention transformers," *Scientific Reports*, vol. 15, p. 28718, 2025.

[11] M. Govindarajan and R. Chandrasekaran, "Intrusion detection using k-nearest neighbor," in *Proc. 1st Int. Conf. Advanced Computing*, pp. 13–20, 2009.

[12] S. Wang, "Research of intrusion detection based on an improved K-Means algorithm," in *Proc. 2nd Int. Conf. Innovations in Bio-inspired Computing and Applications*, pp. 274–276, 2011.

[13] M. Eslamnezhad and A. Y. Varjani, "Intrusion detection based on MinMax K-Means clustering," in *Proc. 7th Int. Symposium on Telecommunications (IST)*, pp. 804–808, 2014.

[14] H. A. Ahmed, A. Hameed, and N. Z. Bawany, "Network intrusion detection using oversampling technique and machine learning algorithms," *PeerJ Computer Science*, vol. 8, p. e820, 2022.

[15] R. Tahri, Y. Balouki, A. Jarrar, and A. Lasbahani, "Intrusion detection system using machine learning algorithms," in *ITM Web of Conferences*, vol. 46, p. 02003, 2022.

[16] N. S. Bhati and M. Khari, "A new intrusion detection scheme using CatBoost classifier," in *Proc. 1st EAI Int. Conf. FoNeS-IoT 2020*, pp. 169–176, 2021.

[17] M. H. Kabir *et al.*, "Network intrusion detection using UNSW-NB15 dataset: Stacking machine learning based approach," in *Proc. Int. Conf. Advancement in Electrical and Electronic Engineering (ICAEEE)*, pp. 1–6, 2022.

[18] N. S. Bhati and M. Khari, "An ensemble model for network intrusion detection using AdaBoost, Random Forest and Logistic Regression," in *Applications of Artificial Intelligence and Machine Learning: Select Proceedings of ICAAAIML 2021*, pp. 777–789, 2022.

[19] J. Kim, J. Kim, H. Kim, M. Shim, and E. Choi, "CNN-based network intrusion detection against denial-of-service attacks," *Electronics*, vol. 9, p. 916, 2020.

[20] V. E. Adeyemo *et al.*, "Ensemble and deep-learning methods for two-class and multi-attack anomaly intrusion detection: An empirical study," *Int. Journal of Advanced Computer Science and Applications*, vol. 10, 2019.

[21] A. Aleesa, M. Younis, A. A. Mohammed, and N. Sahar, "Deep-intrusion detection system with enhanced UNSW-NB15 dataset based on deep learning techniques," *Journal of Engineering Science and Technology*, vol. 16, pp. 711–727, 2021.

[22] S. Choudhary and N. Kesswani, "Analysis of KDD-Cup'99, NSL-KDD and UNSW-NB15 datasets using deep learning in IoT," *Procedia Computer Science*, vol. 167, pp. 1561–1573, 2020.

[23] S. Al and M. Dener, "STL-HDL: A new hybrid network intrusion detection system for imbalanced dataset on big data environment," *Computers & Security*, vol. 110, p. 102435, 2021.

[24] N. U. Ain, M. Sardaraz, M. Tahir, M. W. Abo Elsoud, and A. Alourani, "Securing IoT networks against DDoS attacks: A hybrid deep learning approach," *Sensors*, vol. 25, p. 1346, 2025.

[25] Y.-G. Yang, H.-M. Fu, S. Gao, Y.-H. Zhou, and W.-M. Shi, "Intrusion detection: A model based on the improved vision transformer," *Transactions on Emerging Telecommunications Technologies*, vol. 33, p. e4522, 2022.

[26] T. Jiang, X. Fu, and M. Wang, "BBO-CFAT: Network intrusion detection model based on BBO algorithm and hierarchical transformer," *IEEE Access*, 2024.
[27] Y. Liu and L. Wu, "Intrusion detection model based on improved transformer," *Applied Sciences*, vol. 13, p. 6251, 2023.
[28] Y. Yin *et al.*, "IGRF-RFE: A hybrid feature selection method for MLP-based network intrusion detection on UNSW-NB15 dataset," *Journal of Big Data*, vol. 10, p. 15, 2023.
[29] F. Ullah, S. Ullah, G. Srivastava, and J. C.-W. Lin, "IDS-INT: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic," *Digital Communications and Networks*, vol. 10, pp. 190–204, 2024.
[30] Y. Li, X. Peng, J. Zhang, Z. Li, and M. Wen, "DCT-GAN: Dilated convolutional transformer-based GAN for time series anomaly detection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, pp. 3632–3644, 2021.
[31] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 4765–4774, 2017.
[32] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
[33] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.
[34] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.
[35] Microsoft, "ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator," https://onnxruntime.ai, 2018.
[36] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.